Health-Check Software für die Dienste der Firma

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Elias Brandstätter
Zoe Emily Öllinger

Betreuer:
Prof. DI Christian Aberger

Projektpartner:
hello again GmbH

Leonding, 7. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 7. April 2026 Elias Brandstätter & Zoe Emily Öllinger

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Ausgangslage 1

1.2 Problemstellung 1

1.3 Ziel 1

2 Umfeldanalyse **2**

3 Technologien **3**

3.1 Overview 3

3.2 Frontend 3

3.3 Backend 3

3.4 Cloud & Datenerhaltung 9

3.5 Externe Schnittstellen 9

4 Umsetzung **10**

4.1 Systemarchitektur 10

4.2 Backend-Implementierung 13

4.3 Integration externer Systeme 13

4.4 Datenmanagement 13

4.5 Fehlerbehandlung und Logging 13

4.6 Sicherheit und Zugriff 13

5 Zusammenfassung **14**

Literaturverzeichnis **VI**

Abbildungsverzeichnis **VII**

Tabellenverzeichnis **VIII**

Quellcodeverzeichnis **IX**

1 Einleitung

1.1 Ausgangslage

Hello Again entwickelt Kundenbindungs-Apps für kleine und mittelständige Unternehmen, die über verschiedene Google- und Apple-Developer-Accounts veröffentlicht werden.

1.2 Problemstellung

Häufig treten Probleme auf, wenn Verifizierungen fehlen oder Konfigurationen fehlerhaft sind. Um solche Probleme frühzeitig zu erkennen und zu beheben, soll ein Health Monitoring System entwickelt werden.

1.3 Ziel

Ziel ist die Entwicklung eines Dashboards, das den Status aller Kunden-Apps in Bezug auf die zugehörigen Developer Accounts anzeigt. Es kategorisiert Apps in „Healthy“, „Needs Attention“ und „Critical“ und führt durch einen strukturierten Fehlerbehebungsprozess. Das System ermöglicht das Filtern nach Fehlermeldungen und Parametern, um gezielte Aktionen einzuleiten.

2 Umfeldanalyse

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula. Citing [1] properly.

Was ist eine GUID? Eine GUID kollidiert nicht gerne.

Kabellose Technologien sind in abgelegenen Gebieten wichtig [2].

3 Technologien

3.1 Overview

Allgemeines und Bild über den Tech-Stack...

3.2 Frontend

3.2.1 Vue.js

3.2.2 ...

3.3 Backend

3.3.1 Python

Das Backend des Health-Monitoring-Systems wurde mit der Programmiersprache Python implementiert. Die Entscheidung für Python wurde aus mehreren Gründen getroffen. Einerseits entspricht Python den internen Entwicklungsrichtlinien des Unternehmens, andererseits zeichnet sich Python durch eine hohe Lesbarkeit und eine klare Syntax aus, wodurch sich komplexe Logik übersichtlich implementieren lässt.

Darüber hinaus bietet Python ein umfangreiches Ökosystem an Bibliotheken für Webentwicklung, Datenanalyse sowie Cloud-Integration. Besonders im Kontext dieses Projektes ist die Integration mit Cloud-Diensten wie Google BigQuery von großer Bedeutung. Python stellt hierfür offizielle Clientbibliotheken bereit, wodurch Datenbankabfragen und API-Integrationen effizient umgesetzt werden können.

Ein weiterer Vorteil von Python liegt in seiner starken Community und der kontinuierlichen Weiterentwicklung der Sprache. Dadurch stehen umfangreiche Dokumentationen und Best Practices zur Verfügung, was die Wartbarkeit und langfristige Weiterentwicklung des Systems unterstützt.

Language Background

Python ist eine interpretierte, objektorientierte und hochgradig abstrahierte Programmiersprache, die ursprünglich von Guido van Rossum entwickelt wurde und erstmals im Jahr 1991 veröffentlicht wurde.

Das Hauptziel von Python war es, eine Programmiersprache zu schaffen, die sowohl einfach zu lesen als auch effizient zu verwenden ist. Dieses Ziel wird durch eine klare und minimalistische Syntax erreicht, die sich stark an natürlicher Sprache orientiert. Dadurch wird der Code leichter verständlich und kann schneller von anderen Entwicklern nachvollzogen werden.

Im Gegensatz zu kompilierten Sprachen wird Python in der Regel interpretiert ausgeführt. Dies ermöglicht eine schnellere Entwicklung, da Änderungen am Code unmittelbar getestet werden können. Für die Entwicklung von Web-Backends und Datenverarbeitungssystemen ist diese Flexibilität besonders vorteilhaft.

Einsatz im Backend

Im Rahmen dieses Projekts übernimmt Python mehrere zentrale Aufgaben innerhalb der Backend-Architektur:

- Implementierung der REST-API
- Kommunikation mit externen Schnittstellen
- Verarbeitung und Aggregation von Monitoring-Daten
- Datenabfragen aus BigQuery
- Logik zur Statusanalyse und Fehlererkennung

Besonders wichtig ist die Fähigkeit von Python, verschiedene externe Systeme über API-Schnittstellen anzusprechen. Im Health-Monitoring-System werden beispielsweise Daten aus mehreren Plattformen verarbeitet, darunter App-Store-Systeme, CRM-Systeme und Projektmanagement-Tools.

Die hohe Flexibilität der Sprache ermöglicht es, diese unterschiedlichen Datenquellen in einer zentralen Backend-Logik zusammenzuführen und anschließend strukturiert an das Frontend weiterzugeben.

3.3.2 FastAPI

Für die Implementierung der Backend-Schnittstellen wurde das moderne Python-Framework FastAPI eingesetzt. FastAPI ist ein leistungsfähiges Framework zur Entwicklung von Web-APIs und basiert auf dem ASGI-Standard (Asynchronous Server Gateway Interface).

Der Hauptzweck von FastAPI besteht darin, eine schnelle und strukturierte Entwicklung von REST-Schnittstellen zu ermöglichen. Dabei kombiniert das Framework hohe Performance mit einer klaren API-Definition und automatischer Dokumentation.

Im Health-Monitoring-System wird FastAPI verwendet, um verschiedene Endpunkte bereitzustellen, über die das Frontend Daten über den Status der Kunden-Apps abrufen kann. Darüber hinaus ermöglicht das Framework die Integration externer APIs sowie die Verarbeitung komplexer Datenstrukturen.

Framework Background

FastAPI wurde von Sebastián Ramírez entwickelt und erstmals im Jahr 2018 veröffentlicht. Das Framework basiert auf modernen Python-Features wie Typannotationen und asynchroner Programmierung.

Ein wesentliches Merkmal von FastAPI ist die Verwendung von Python-Typdefinitionen zur automatischen Validierung von Daten. Dadurch können Request-Parameter und Response-Objekte klar definiert werden, was sowohl die Sicherheit als auch die Stabilität der API erhöht.

Zusätzlich generiert FastAPI automatisch eine OpenAPI-Spezifikation, aus der eine interaktive API-Dokumentation erstellt wird. Diese Dokumentation kann von Entwicklern direkt im Browser verwendet werden, um API-Endpunkte zu testen und deren Struktur zu verstehen.

REST-API Architektur

Das Backend des Health-Monitoring-Systems folgt einer klassischen REST-Architektur. Dabei stellt das System verschiedene Endpunkte zur Verfügung, über die Daten abgefragt oder Aktionen ausgelöst werden können.

API-Endpunkte im Projekt sind beispielsweise:

- Abrufen aller Health-Checks
- Anzeigen der Details eines bestimmten Kunden
- Erstellen von Tasks oder Kommentaren
- Abfragen von Ticket-Informationen

Diese Endpunkte werden in FastAPI über Route-Deklarationen definiert. Jede Route beschreibt eine spezifische HTTP-Operation, beispielsweise GET oder POST.

Die Verwendung einer REST-basierten Architektur ermöglicht eine klare Trennung zwischen Frontend und Backend, wodurch beide Systeme unabhängig voneinander weiterentwickelt werden können.

Performance und Skalierbarkeit

FastAPI gehört zu den performantesten Python-Webframeworks. Durch die Unterstützung asynchroner Programmierung können mehrere Anfragen gleichzeitig verarbeitet werden, ohne dass das System blockiert wird.

Gerade im Kontext dieses Projekts ist dies besonders wichtig, da mehrere externe APIs abgefragt werden müssen. Diese Abfragen können parallel ausgeführt werden, wodurch die Gesamtantwortzeit reduziert wird.

Die Kombination aus asynchroner Verarbeitung und einem effizienten Server-Framework ermöglicht eine gute Skalierbarkeit des Systems.

3.3.3 Uvicorn

Zur Ausführung der FastAPI-Anwendung wird der ASGI-Server Uvicorn verwendet. Uvicorn ist ein leichtgewichtiger und performanter Webserver für Python-Anwendungen, die auf dem ASGI-Standard basieren.

Der Server übernimmt die Aufgabe, eingehende HTTP-Requests zu empfangen und diese an die FastAPI-Anwendung weiterzuleiten. Nachdem die Anfrage verarbeitet wurde, wird die generierte Response an den Client zurückgesendet.

ASGI Standard

Uvicorn basiert auf dem ASGI-Standard (Asynchronous Server Gateway Interface). Dieser Standard stellt eine moderne Alternative zum älteren WSGI-Standard dar und ermöglicht die Verarbeitung asynchroner Anwendungen.

Der Vorteil von ASGI besteht darin, dass mehrere parallele Anfragen effizient verarbeitet werden können. Während bei klassischen synchronen Webservern jede Anfrage einen eigenen Thread blockiert, können asynchrone Server mehrere Operationen gleichzeitig verwalten.

Dies ist besonders relevant für Anwendungen, die häufig externe APIs aufrufen oder auf Datenbankabfragen warten.

Einsatz im Projekt

Im Health-Monitoring-System fungiert Uvicorn als Laufzeitumgebung für die FastAPI-Anwendung. Der Server startet die API und stellt sie über einen HTTP-Port zur Verfügung.

Typische Aufgaben des Servers sind:

- Entgegennahme von HTTP-Requests
- Weiterleitung der Requests an FastAPI
- Verarbeitung von Response-Daten
- Verwaltung paralleler Anfragen

Durch die hohe Performance von Uvicorn kann das Backend auch bei mehreren gleichzeitigen Anfragen effizient arbeiten.

3.3.4 Pydantic

Für die Datenvalidierung und Modellierung von Datenstrukturen wird die Python-Bibliothek Pydantic eingesetzt. Pydantic ermöglicht eine strukturierte Definition von Datenmodellen, die automatisch validiert und konvertiert werden können.

Diese Modelle werden insbesondere bei der Verarbeitung von API-Requests und Responses verwendet. Dadurch kann sichergestellt werden, dass alle übermittelten Daten dem erwarteten Format entsprechen.

Data Validation

Ein zentrales Feature von Pydantic ist die automatische Validierung von Daten. Wenn ein Client eine Anfrage an die API sendet, überprüft Pydantic automatisch, ob alle erforderlichen Felder vorhanden sind und ob diese dem definierten Datentyp entsprechen.

Beispielsweise kann ein API-Request definieren, dass ein bestimmtes Feld eine Zeichenkette sein muss oder dass ein Datum in einem bestimmten Format übergeben werden muss. Falls diese Anforderungen nicht erfüllt werden, wird automatisch eine Fehlermeldung generiert.

Diese Validierung erhöht die Stabilität der Anwendung erheblich, da fehlerhafte Eingaben frühzeitig erkannt werden.

Datenmodelle

Pydantic verwendet Model-Klassen, um Datenstrukturen zu definieren. Diese Modelle beschreiben die Struktur von JSON-Objekten, die über die API übertragen werden.

Beispielsweise können Modelle für folgende Daten definiert werden:

- Health-Check-Ergebnisse
- Kundeninformationen
- Task-Objekte
- Ticket-Daten

Durch diese strukturierte Definition wird sichergestellt, dass alle Daten innerhalb des Systems ein einheitliches Format besitzen.

Vorteile für API-Entwicklung

Der Einsatz von Pydantic bietet mehrere Vorteile für die Entwicklung von APIs:

- Automatische Datenvalidierung
- Klare Definition von Datenstrukturen
- Integration mit FastAPI
- Automatische Generierung von API-Schemas

3.4 Cloud & Datenerhaltung

3.4.1 Google BigQuery

3.4.2 Google Cloud Platform

3.4.3 SQL

3.5 Externe Schnittstellen

3.5.1 Asana API

3.5.2 HubSpot API

3.5.3 Google Play Developer API

3.5.4 App Store Connect API

4 Umsetzung

4.1 Systemarchitektur

Die Systemarchitektur des Health-Monitoring-Systems folgt einer mehrschichtigen Architektur, die aus einem Frontend, einem Backend sowie mehreren externen Datenquellen besteht. Ziel dieser Architektur ist es, Daten aus verschiedenen Plattformen zentral zu sammeln, zu analysieren und anschließend in einem übersichtlichen Dashboard darzustellen.

Durch die klare Trennung der einzelnen Komponenten wird eine modulare und wartbare Systemstruktur erreicht. Jede Schicht des Systems erfüllt eine spezifische Aufgabe und kommuniziert über definierte Schnittstellen mit den anderen Komponenten.

4.1.1 Gesamtübersicht der Architektur

Das System besteht aus vier zentralen Ebenen:

- Frontend (Dashboard)
- Backend (API und Datenverarbeitung)
- Cloud- und Datenebene
- Externe Schnittstellen

Das Frontend dient als Benutzeroberfläche für Mitarbeiter des Unternehmens und ermöglicht die Anzeige sowie Analyse des Status aller Kunden-Apps. Das Backend stellt eine REST-API bereit, über die das Frontend Daten abrufen kann. Gleichzeitig übernimmt das Backend die Verarbeitung und Aggregation der Daten aus verschiedenen Quellen.

Die Cloud- und Datenebene stellt die zentrale Datenbasis dar, während externe Schnittstellen zusätzliche Informationen aus Drittplattformen liefern.

4.1.2 Frontend-Schicht

4.1.3 Backend-Schicht

Das Backend stellt die zentrale Logik des Systems dar und fungiert als Vermittler zwischen Frontend, Datenbank und externen Schnittstellen. Die Backend-Komponente wurde mit FastAPI entwickelt und läuft auf dem ASGI-Server Uvicorn.

Die wichtigsten Aufgaben des Backends sind:

- Bereitstellung einer REST-API für das Frontend
- Abrufen und Verarbeiten von Daten aus Google BigQuery
- Integration externer APIs
- Durchführung von Statusanalysen
- Erkennung von Statusänderungen

Das Backend aggregiert Daten aus mehreren Quellen und stellt sie in strukturierter Form bereit. Diese Daten werden anschließend vom Frontend dargestellt.

Ein wichtiger Bestandteil der Backend-Logik ist die Analyse der Statuswerte der einzelnen Apps. Hierbei werden verschiedene Health-Checks durchgeführt und deren Ergebnisse miteinander kombiniert, um einen Gesamtstatus zu bestimmen.

4.1.4 Daten- und Cloud-Schicht

Die zentrale Datenbasis des Systems befindet sich in der Cloud-Infrastruktur der Google Cloud Platform.

Die Monitoring-Daten werden in Google BigQuery gespeichert und über SQL-Abfragen vom Backend abgerufen. BigQuery ermöglicht eine effiziente Verarbeitung großer Datenmengen und eignet sich besonders für analytische Abfragen.

Zusätzlich werden Snapshots der Statusdaten lokal gespeichert, um Veränderungen im Status der Apps über einen bestimmten Zeitraum hinweg nachvollziehen zu können. Dieses Snapshot-System ermöglicht es, Statusänderungen zwischen zwei Zeitpunkten automatisch zu erkennen.

4.1.5 Integration externer Systeme

Ein vitaler Bestandteil der Systemarchitektur ist die Integration externer Plattformen. Das Monitoring-System greift auf mehrere APIs zu, um Informationen über App-Status, Support-Tickets und Aufgaben zu erhalten.

Die wichtigsten integrierten Systeme sind:

- Asana: zur Verwaltung von Aufgaben
- HubSpot: zur Bereitstellung von Support-Tickets
- Google Play Developer API: zur Überprüfung von Android-App-Status
- App Store Connect API: zur Überprüfung von iOS-App-Status

4.1.6 Datenfluss im System

Der typische Datenfluss innerhalb des Systems lässt sich in mehreren Schritten beschreiben:

1. Das Frontend sendet eine Anfrage an das Backend, beispielsweise zur Anzeige aller Health-Checks.
2. Das Backend führt SQL-Abfragen auf BigQuery aus und ruft aktuelle Monitoring-Daten ab.
3. Zusätzlich werden Informationen aus externen APIs wie Asana oder HubSpot abgefragt.
4. Die Daten werden im Backend aggregiert und analysiert.
5. Das Backend sendet die aufbereiteten Daten als JSON-Antwort an das Frontend zurück.
6. Frontend visualisiert die Ergebnisse im Dashboard.

Durch diese Architektur wird sichergestellt, dass alle Daten zentral verarbeitet und konsistent dargestellt werden.

4.1.7 Vorteile der gewählten Architektur

Die gewählte Systemarchitektur bietet mehrere Vorteile:

- Modularität: einzelne Komponenten können unabhängig weiterentwickelt werden

- Skalierbarkeit: Cloud-Infrastruktur ermöglicht Erweiterung bei steigender Datenmenge
- Wartbarkeit: klare Trennung zwischen Frontend, Backend und Datenebene
- Flexibilität: neue APIs oder Datenquellen können einfach integriert werden

Diese Architektur bildet somit eine stabile Grundlage für den Betrieb und die zukünftige Erweiterung des Health-Monitoring-Systems.

4.2 Backend-Implementierung

4.3 Integration externer Systeme

4.4 Datenmanagement

4.5 Fehlerbehandlung und Logging

4.6 Sicherheit und Zugriff

5 Zusammenfassung

Aufzählungen:

- Itemize Level 1
  - Itemize Level 2
    - Itemize Level 3 (vermeiden)

1. Enumerate Level 1
   a. Enumerate Level 2
      i. Enumerate Level 3 (vermeiden)

Desc Level 1

Desc Level 2 (vermeiden)

Desc Level 3 (vermeiden)

Literaturverzeichnis

[1] P. Rechenberg, G. Pomberger *et al.*, *Informatik Handbuch*, 4. Aufl. München – Wien: Hanser Verlag, 2006.

[2] Association for Progressive Communications, „Wireless technology is irreplaceable for providing access in remote and scarcely populated regions," 2006, letzter Zugriff am 23.05.2021. Online verfügbar: http://www.apc.org/en/news/strategic/world/wireless-technology-irreplaceable-providing-access

Abbildungsverzeichnis

Tabellenverzeichnis

Quellcodeverzeichnis

Anhang